

Historias de Usuario — Proyecto 2

Convenciones usadas

- **Actor:** Cliente / Empleado / Administrador
- **Entradas (Consola):** datos solicitados en UI
- **Validaciones:** reglas que se deben cumplir
- **Salidas/Resultados:** mensajes + cambios persistentes
- **Clases involucradas:** clases/métodos exactos del UML.

0) Historias transversales (aplican a los 3 roles)

HU-T0 — Autenticación de usuario (Login)

Estimación: *TBD story points*

Plantilla

- **As a (Como):** usuario del sistema (Cliente / Empleado / Administrador)
- **I want (Quiero):** iniciar sesión con mis credenciales
- **So that (Para):** acceder al menú y funciones correspondientes a mi rol

Descripción (contexto)

Esta historia define el **punto de entrada obligatorio** al sistema: **nadie puede operar funciones** (compras, préstamos, torneos, turnos) sin autenticarse. El propósito es asegurar que la consola muestre opciones según el tipo real de Usuario (Cliente/Empleado/Administrador) y que las acciones queden asociadas al usuario autenticado. En tu UML esto se soporta con `Cafeteria.login(...)`, `GestorUsuarios.autenticar(...)` y la sesión del Usuario. ,

Precondiciones

- El usuario existe en el sistema (registrado en la lista de usuarios).
- Datos cargados desde persistencia al iniciar el programa. ,

Entradas (Consola)

- login: String
- password: String

Validaciones

- No vacío, longitud mínima (ej. ≥ 4).
- Credenciales correctas.

Flujo principal

1. El usuario ingresa login y password.
2. El sistema autentica usando `Cafeteria.login(login, pass)` o `GestorUsuarios.autenticar(login, password)`.
3. Se identifica el tipo (Cliente/Empleado/Administrador) y se redirige al menú correspondiente.

Salidas/Resultados

- **Éxito:** "Bienvenido {nombre}" y menú del rol.
- **Fallo:** "Credenciales inválidas" y se reintenta.

Clases involucradas (UML)

- `Cafeteria.login(login: String, pass: String): Usuario`
- `GestorUsuarios.autenticar(login: String, password: String): Usuario`
- `Usuario.iniciarSesion(login: String, pass: String): boolean`

Criterios de aceptación

- Si el usuario no existe o la contraseña no coincide, no inicia sesión.
- Si inicia sesión, se habilitan solo las opciones del rol.

1) Historias de Usuario — CLIENTE

HU-C1 — Registro autónomo de cliente

Estimación: *TBD story points*

Plantilla

- **As a:** Cliente
- **I want:** registrarme por mi cuenta en la aplicación
- **So that:** poder autenticarme y usar las funciones del sistema (compras, préstamos, torneos)

Descripción (contexto)

Esta historia implementa el requisito del enunciado: **los clientes sí pueden registrarse solos**, a diferencia de los empleados. El registro crea un objeto `Cliente` con sus atributos (edad/segmento, puntos, etc.) y lo deja disponible para iniciar sesión inmediatamente. El control de unicidad del login se hace con `GestorUsuarios.existeUsuario(...)` y la creación se hace con `GestorUsuarios.registrarCliente(...)`. El resultado debe persistirse usando `GestorPersistencia` al finalizar la ejecución. ,

Entradas (Consola)

- login: String
- password: String
- nombre: String
- esNino: boolean
- esJoven: boolean

Validaciones

- login único (no puede existir ya) → `GestorUsuarios.existeUsuario(login)` ,
- password con reglas mínimas (longitud, no vacío).
- consistencia de flags: si `esNino=true` y `esJoven=true`, permitir o forzar una regla (recomendado: **no ambos**).

Flujo principal

1. Cliente elige "Registrarse".
2. Ingresa datos.
3. Sistema verifica login único.
4. Se crea el cliente con `GestorUsuarios.registrarCliente(...)` .,
5. Se confirma registro y se ofrece iniciar sesión.

Salidas/Resultados

- "Cliente registrado exitosamente".
- Usuario almacenado en el catálogo de usuarios (persistir al final del programa).

Clases involucradas (UML)

- `GestorUsuarios.registrarCliente(login, password, nombre, esNino, esJoven): Cliente`
- Cliente atributos: `esNiño`, `esJoven`, `puntosFidelidad...`
- `GestorPersistencia.guardarUsuarios(lista: List<Usuario>)` al terminar

Criterios de aceptación

- No permite registrar si el login ya existe.
- Cliente queda disponible para login inmediatamente.

HU-C2 — Marcar / desmarcar juego favorito (fanático)

Estimación: *TBD story points*

Plantilla

- **As a:** Cliente
- **I want:** marcar o desmarcar juegos como favoritos
- **So that:** ser considerado fanático y tener prioridad en cupos reservados (20%) en torneos

Descripción (contexto)

Esta historia habilita la regla “**fanáticos**” para torneos: un fanático es un usuario que tiene ese juego dentro de su lista `juegosFavoritos`. En tu UML, `Usuario` permite `agregarFavorito(j)` y `eliminarFavorito(j)`, y la relación `Cliente – juegosFavoritos – Juego` ya existe. Luego, `Torneos` usa ese dato con `Torneo.esFanatico(u)` para decidir si usa cupos reservados primero.

Entradas

- `nombreJuego: String` (o selección de lista)

Validaciones

- Juego existe → `Cafeteria.buscarJuego(nombre)`
- Si ya es favorito, no duplicar.

Flujo principal

1. Cliente busca/selecciona juego.
2. El sistema ejecuta `Usuario.agregarFavorito(j)` o `Usuario.eliminarFavorito(j)`.

Salidas

- “Juego agregado a favoritos” / “Juego eliminado de favoritos”.

Clases involucradas

- Usuario.agregarFavorito(j: Juego) / Usuario.eliminarFavorito(j: Juego)
- Relación Cliente – juegosFavoritos 0..* – Juego ,

Criterios de aceptación

- Favoritos se usan para determinar Torneo.esFanatico(u) en inscripciones.

HU-C3 — Consultar catálogo de torneos por día

Estimación: *TBD story points*

Plantilla

- **As a:** Cliente
- **I want:** consultar torneos disponibles filtrados por día de la semana
- **So that:** decidir en cuáles inscribirme según horario y cupos

Descripción (contexto)

Esta historia permite que el cliente explore el “**catálogo de torneos**” que administra el sistema a través de GestorTorneos. Como tu UML define el atributo día: DiaSemana dentro de Torneo y el método GestorTorneos.getTorneos(día), esta funcionalidad es directa y sirve como base para inscripciones. Además, el listado debe mostrar cupos disponibles (reservados/regulares) con los métodos del Torneo. ,

Entradas

- día: DiaSemana (selección 1..7)

Validaciones

- día válido (enum).

Flujo principal

1. Cliente selecciona “Ver torneos”.
2. Ingresa día.
3. Sistema usa GestorTorneos.getTorneos(día): List<Torneo>. ,

Salidas

- Lista con: nombre, juego, hora, estado, cupos disponibles.
 - cupos disponibles salen de `Torneo.cuposDisponiblesReservados()` y `Torneo.cuposDisponiblesRegulares()`.

Clases involucradas

- `GestorTorneos`, `Torneo`, `EstadoTorneo`, `DiaSemana`,

Criterios de aceptación

- Solo muestra Programados/En_Curso (si decides ocultar Finalizados/Cancelados, lo documentas como regla de UI).

HU-C4 — Inscribirse a un torneo (hasta 3 cupos)

Estimación: *TBD story points*

Plantilla

- **As a:** Cliente
- **I want:** inscribirme a un torneo tomando entre 1 y 3 cupos
- **So that:** asegurar espacios para mí y/o acompañantes y participar el día del evento

Descripción (contexto)

Esta historia implementa el corazón del módulo Torneos: crear una `InscripcionTorneo` asociada al `Usuario` y al `Torneo`, respetando reglas críticas: máximo 3 cupos por usuario, cupos reservados para fanáticos (20% con `ceil`), y comportamiento de cupos reservados vs regulares. Tu UML ya modela esto explícitamente en `Torneo` (regla `MaxCuposPorUsuario=3`, métodos `puedeInscribirse`, `inscribir`, `esFanatico`) y en `InscripcionTorneo` (`cantidadCupos`, `cuposReservadosUsados`, `cuposRegularesUsados`).

Entradas

- `torneoId`: `String`
- `cantidadCupos`: `int` (1..3)

Validaciones (todas obligatorias)

- `cantidadCupos` $\in$ [1..3] y respeta `reglaMaxCuposPorUsuario = 3`.
- El torneo existe y está en estado Programado (regla recomendada).

- Por par (Usuario, Torneo) solo existe 0..1 inscripción (no duplicar). ,
- Hay cupos disponibles (reservados si es fanático o regulares si no).
- Regla 20% reservado: $\text{cupoReservadoFanaticos} = \text{ceil}(0.2 * \text{cupoTotal})$. ,
- Validar cupo total vs copias de préstamo del juego:
`GestorTorneos.validarCupoVsCopiasPrestamo(juego, cupoTotal)`. ,

Flujo principal

1. Cliente elige torneo y cantidad.
2. Sistema llama `GestorTorneos.inscribir(usuario, torneoId, cantidadCupos)`. ,
3. Dentro de la lógica (según tus métodos en Torneo):
 - a. `Torneo.puedeInscribirse(u, cantidad)`
 - b. `Torneo.inscribir(u, cantidad)`
4. Se crea `InscripcionTorneo` con:
 - a. `cantidadCupos`
 - b. `cuposReservadosUsados / cuposRegularesUsados` (según disponibilidad y si es fanático) ,

Salidas

- "Inscripción exitosa. Cupos reservados usados: X, regulares usados: Y".
- O error: "No hay cupos", "Excede máximo 3", "Ya estabas inscrito".

Clases involucradas

- `GestorTorneos.inscribir(...)`
- `Torneo.inscribir, Torneo.esFanatico`
- `InscripcionTorneo`

Criterios de aceptación

- Fanático usa primero cupos reservados; si se agotan, usa regulares. ,
- Nunca puede tomar más de 3 cupos.

HU-C5 — Desinscribirse de un torneo (libera todos los cupos)

Estimación: *TBD story points*

Plantilla

- **As a:** Cliente
- **I want:** desinscribirme de un torneo
- **So that:** liberar todos mis cupos y dejar disponible el espacio para otros participantes

Descripción (contexto)

Esta historia complementa HU-C4: así como un usuario puede tomar cupos, también debe poder **liberarlos completamente**. El enunciado exige que al desinscribir se eliminen **todos los cupos tomados**, lo cual encaja con tu modelo porque `Torneo.desinscribir(u)` elimina la inscripción (0..1 por usuario/torneo) y por tanto libera el total. En consola se espera un flujo simple: escoger torneo, confirmar y ejecutar. ,

Entradas

- `torneoId: String`

Validaciones

- Debe existir inscripción del usuario en ese torneo.

Flujo

1. Cliente elige "Desinscribirse".
2. Sistema llama `GestorTorneos.desinscribir(usuario, torneoId).`
3. `Torneo.desinscribir(u)` libera cupos y elimina inscripción.

Salidas

- "Desinscripción exitosa. Cupos liberados: N".
- O "No estás inscrito en este torneo".

Clases involucradas

- `GestorTorneos.desinscribir` y `Torneo.desinscribir`

Criterios de aceptación

- Elimina **todos los cupos tomados** (inscripción completa).

HU-C6 — Realizar compra y aplicar bono de torneo amistoso (no acumulable)

Estimación: *TBD story points*

Plantilla

- **As a:** Cliente
- **I want:** realizar una compra y aplicar un bono de torneo amistoso si lo tengo disponible

- **So that:** obtener el beneficio ganado sin poder combinarlo con otros descuentos o puntos

Descripción (contexto)

Esta historia conecta el módulo de Torneos con Ventas. Si el usuario gana un TorneoAmistoso, obtiene un BonoTorneoAmistoso. Luego, al comprar, puede aplicar ese bono usando `Venta.aplicarBono(bono)`. Tu UML también incluye la nota clave: **no acumulable**, es decir, una venta no debería aplicar bono si ya aplicó otros descuentos/puntos. Esto requiere validación de UI y/o regla de negocio en Venta. ,

Entradas

- Lista de productos/ítems: `items: List<ItemVenta>` (selección + cantidades)
- `codigoDesc: String` (opcional)
- Opción: "Aplicar bono" (si tiene)

Validaciones

- Productos existen.
- Si aplica bono:
 - Venta no debe tener otro descuento/puntos aplicados (nota UML: no acumulable). ,
 - Bono disponible (estado no usado).

Flujo

1. Cliente arma carrito (items).
2. Ejecuta `Cliente.realizarCompra(items, codigoDesc): Venta. ,`
3. Si decide aplicar bono, usar `Venta.aplicarBono(bono: BonoTorneoAmistoso). ,`
4. Venta calcula subtotal/impuestos/total.

Salidas

- Factura con total y detalle de descuento por bono.
- Bono pasa a estado "USADO" (si decides modelar transición; ya tienes `fechaUsado`).

Clases involucradas

- `Cliente.realizarCompra`, `Venta.aplicarBono`, `BonoTorneoAmistoso`

Criterios de aceptación

- Si se aplicó bono, no se permite aplicar otros descuentos/puntos en esa venta

2) Historias de Usuario — EMPLEADO (Mesero / Cocinero)

HU-E2 — Consultar turnos asignados

Estimación: *TBD story points*

Plantilla

- **As a:** Empleado
- **I want:** consultar mis turnos asignados
- **So that:** saber cuándo trabajo y validar si puedo inscribirme a torneos en esas fechas

Descripción (contexto)

Esta historia es esencial porque el enunciado dice que un empleado **no puede inscribirse a torneos si está cubriendo turno** el día/hora del torneo. Para que el empleado decida correctamente, primero necesita visualizar sus `DiaTurno` asignados. Tu UML lo soporta mediante `Empleado.consultarDiaAsignado()` y los objetos `DiaTurno` dentro de la planificación (`Semana`).

Entradas

- (ninguna) o "Ver turnos"

Validaciones

- Sesión activa.

Flujo

1. Empleado elige "Consultar turnos".
2. Sistema llama `Empleado.consultarDiaAsignado(): List<DiaTurno>`.

Salidas

- Lista de días asignados + estado aprobado (`DiaTurno.estaAprobado`).

Clases involucradas

- `Empleado.consultarDiaAsignado()`

- DiaTurno

Criterios

- Muestra 0..* días asignados.

HU-E3 — Solicitar cambio/intercambio de turno

Estimación: *TBD story points*

Plantilla

- **As a:** Empleado
- **I want:** solicitar un cambio o intercambio de turno
- **So that:** ajustar mi disponibilidad laboral sin romper la operación del café

Descripción (contexto)

Esta historia sostiene la operación del café: los empleados pueden necesitar mover turnos, y el Administrador es quien aprueba/rechaza. En tu UML, el empleado crea una *SolicitudTurno* (ya sea cambio o intercambio) usando los métodos *solicitarCambioTurno* y *solicitarIntercambioTurno*. Luego (HU-A7) el Admin la gestiona. Esto también impacta la restricción de inscripción a torneos (HU-E4). ,

Entradas

- Para cambio: dia: DiaSemana
- Para intercambio: otroEmpleadoLogin + dia

Validaciones

- Día válido.
- "Otro empleado" existe (si intercambio).

Flujo

1. Empleado crea solicitud con:
 - a. `Empleado.solicitarCambioTurno(dia)` o
 - b. `Empleado.solicitarIntercambioTurno(otro, dia)` ,
2. Se genera *SolicitudTurno* en estado inicial (ej. "Pendiente").

Salidas

- "Solicitud creada con ID/estado".

Clases involucradas

- Empleado + SolicitudTurno ,

HU-E4 — Inscribirse a torneo (respetando turnos)

Estimación: *TBD story points*

Plantilla

- **As a:** Empleado
- **I want:** inscribirme a un torneo cuando no tengo turno asignado en ese horario
- **So that:** participar sin afectar mis responsabilidades laborales y respetar las reglas del torneo

Descripción (contexto)

Esta historia implementa las reglas diferenciadoras del enunciado para empleados:

1. pueden inscribirse **solo si no están de turno** ese día/hora;
2. en torneos competitivos, **no pagan**, pero **no pueden ganar premio metálico**.

Tu UML ya trae un método específico de validación:

`GestorTorneos.validarEmpleadoSinTurno(...)`, y `InscripcionTorneo` tiene banderas para marcar si es empleado y si es elegible a premio metálico. ,

Entradas

- `torneoId: String`
- `cantidadCupos: int (1..3)` (si decides permitir cupos múltiples para empleado; tu UML lo permite por inscripción genérica)

Validaciones clave

- `GestorTorneos.validarEmpleadoSinTurno(empleado, dia, hora) == true.`,
- Máximo 3 cupos.
- Si torneo competitivo:
 - Empleado **gratis** → `montoPagado = 0`, `pagoConfirmado = true/false` según lógica.
 - **No elegible premio metálico** → `elegiblePremioMetalico = false.`,

Flujo

1. Empleado selecciona torneo.
2. Sistema valida turnos con `validarEmpleadoSinTurno(...)`.
3. Inscribe y crea `InscripcionTorneo` con `esEmpleado=true`.

Salidas

- Confirmación o rechazo por turno.

Clases involucradas

- `GestorTorneos.inscribir`, `GestorTorneos.validarEmpleadoSinTurno`
- `InscripcionTorneo` ,

HU-E5 — Desinscribirse de torneo

Estimación: *TBD story points*

Plantilla

- **As a:** Empleado
- **I want:** desinscribirme de un torneo
- **So that:** liberar cupos si ya no puedo/quiero participar o si cambió mi disponibilidad

Descripción (contexto)

Esta historia es equivalente a la del cliente, pero aplicada al empleado. Es importante porque los turnos pueden cambiar (HU-E3 y HU-A7), y un empleado podría quedar luego con conflicto. El flujo de desinscripción en tu UML se hace con el mismo mecanismo `GestorTorneos.desinscribir(usuario, torneoId)` y elimina por completo la inscripción y cupos asociados. ,

Mismo comportamiento que Cliente

- Llama `GestorTorneos.desinscribir(usuario, torneoId)` y elimina cupos.

HU-E6 — Mesero: registrar venta a una mesa

Estimación: *TBD story points*

Plantilla

- **As a:** Mesero
- **I want:** registrar una venta asociada a una mesa
- **So that:** cobrar consumos de clientes y generar el registro de ventas del café

Descripción (contexto)

Aunque el proyecto 2 se enfoca en torneos, tu sistema ya incluye un módulo de venta robusto. Esta historia es clave para asegurar que la consola del empleado cubre operaciones principales. En tu UML, el Mesero registra venta con `registrarVenta(items, mesa)` y luego esa venta se almacena con `GestorVentas.registrarVenta(venta)`. Esto también se conecta indirectamente con HU-C6 (aplicar bono) cuando el comprador es cliente y la venta soporta `aplicarBono.` ,

Entradas

- `idMesa`
- `items` (producto + cantidad)
- `propina` (porcentaje opcional)

Validaciones

- Mesa existe y está ocupada/usable.
- Items válidos.

Flujo

1. Mesero registra venta: `Mesero.registrarVenta(items, mesa): Venta.` ,
2. Venta calcula totales.
3. Se almacena vía `GestorVentas.registrarVenta(venta).` ,

Salidas

- Resumen de la venta + total.

3) Historias de Usuario — ADMINISTRADOR

HU-A1 — Autenticación de administrador

Estimación: *TBD story points*

Plantilla

- **As a:** Administrador
- **I want:** iniciar sesión como administrador
- **So that:** acceder a funciones exclusivas (gestión de empleados, torneos, inventario, aprobaciones)

Descripción (contexto)

Es la misma autenticación base (HU-T0), pero enfatizando que el rol Administrador desbloquea funcionalidades estrictamente administrativas: crear torneos, finalizar torneos, registrar empleados, aprobar solicitudes, etc. En tu UML esto se refleja en la clase Administrador y sus métodos. ,

Cubre HU-T0, con menú de admin.

HU-A2 — Registrar un empleado (Mesero/Cocinero) (obligatorio por enunciado)

Estimación: *TBD story points*

Plantilla

- **As a:** Administrador
- **I want:** registrar empleados (meseros/cocineros) en el sistema
- **So that:** puedan autenticarse y operar funciones del café sin permitir auto-registro

Descripción (contexto)

Esta historia implementa literalmente el requisito del enunciado: **los empleados NO se registran solos**; los crea el Administrador. El objetivo es controlar quién tiene acceso como empleado, asignarle tipo (Mesero o Cocinero) y dejarlo listo para operar en consola. Tu UML tiene `GestorUsuarios.registrarEmpleado(...)`, y el Admin es quien lo usa desde su menú. ,

Entradas (Consola)

- login: String
- password: String
- nombre: String
- tipo: String (Mesero | Cocinero)
- codigoDescuento: String (si aplica al empleado en tu modelo)

Validaciones

- login único → `GestorUsuarios.existeUsuario(login)`
- tipo válido (solo Mesero/Cocinero).
- password reglas mínimas.

Flujo

1. Admin selecciona "Registrar empleado".
2. Ingresa datos.
3. Sistema crea con `GestorUsuarios.registrarEmpleado(login, password, nombre, tipo, codigoDescuento): Empleado.`

Salidas

- "Empleado registrado exitosamente (tipo = Mesero/Cocinero)".

Clases involucradas

- `GestorUsuarios.registrarEmpleado(...)`
- `Empleado` (abstract) + `Mesero`, `Cocinero` ,

Criterios de aceptación

- Empleado puede luego iniciar sesión con sus credenciales.
- No existe registro autónomo de empleado.

HU-A3 — Crear torneo amistoso

Estimación: *TBD story points*

Plantilla

- **As a:** Administrador
- **I want:** crear un torneo amistoso con un juego, día, hora y cupos
- **So that:** atraer clientes sin cobrar entrada y premiar con un bono de descubrimiento

Descripción (contexto)

Esta historia crea torneos "sin tarifa" donde el incentivo es un **bono redimible** (BonoTorneoAmistoso). Debe quedar configurada la regla del 20% de cupos reservados para fanáticos y validarse que el cupo sea viable con copias de préstamo del juego (tu UML lo contempla en `GestorTorneos.validarCupoVsCopiasPrestamo`). La creación se hace con `GestorTorneos.crearTorneoAmistoso(...)` y genera un `TorneoAmistoso` en estado Programado. ,

Entradas

- juego: (nombre o selección)
- dia: DiaSemana
- hora: String (o LocalTime)
- cupoTotal: int
- valorBono: double
- nombreTorneo: String (si lo pides)

Validaciones

- `cupoTotal > 0`
- `valorBono > 0`
- El juego existe.
- Validación cupo vs copias de préstamo:
 - `GestorTorneos.validarCupoVsCopiasPrestamo(juego, cupoTotal)` ,
- Calcular y almacenar cupo reservado fanáticos:
 - `cupoReservadoFanaticos = ceil(0.2*cupoTotal)` (nota UML ya la tienes). ,

Flujo

1. Admin ingresa parámetros.
2. Sistema ejecuta:
 - a. `GestorTorneos.crearTorneoAmistoso(admin, juego, dia, hora, cupoTotal, valorBono): TorneoAmistoso` ,
3. Torneo queda con estado = Programado.

Salidas

- "Torneo amistoso creado con id = ..."
- Mostrar cupos reservados calculados.

Clases involucradas

- `GestorTorneos`, `TorneoAmistoso`, `Torneo`

HU-A4 — Crear torneo competitivo

Estimación: *TBD story points*

Plantilla

- **As a:** Administrador
- **I want:** crear un torneo competitivo con tarifa de entrada
- **So that:** recaudar inscripciones y ofrecer un premio metálico coherente con el pozo recaudado

Descripción (contexto)

Esta historia crea torneos con inscripción paga. Aunque el pago sea tercerizado, tu UML prevé registrar el pago (monto/confirmación). Además, el premio metálico se gestiona con `pozoPremio` y se calcula con `calcularPozo()`. También aplica la excepción de empleados: en competitivo el empleado entra gratis pero no es elegible para premio metálico. Todo esto se deriva del `TorneoCompetitivo` en tu diagrama. ,

Entradas

- juego
- dia
- hora
- cupoTotal
- tarifa: double
- (opcional) porcentajePozoPremio: double

Validaciones

- Todo lo de HU-A3 (cupos total, juego, copias).
- $\text{tarifa} > 0$.

Flujo

1. Admin ingresa parámetros.
2. Sistema ejecuta:
 - a. `GestorTorneos.crearTorneoCompetitivo(admin, juego, dia, hora, cupoTotal, tarifa): TorneoCompetitivo` ,
3. El pozo se calcula (según implementación): `TorneoCompetitivo.calcularPozo()` . ,

Salidas

- "Torneo competitivo creado... Tarifa... Cupos..."

Clases involucradas

- TorneoCompetitivo atributos `tarifaEntrada`, `pozoPremio`, `porcentajePozoPremio`

HU-A5 — Finalizar torneo y registrar ganador (bono o restricciones de premio)

Estimación: *TBD story points*

Plantilla

- **As a:** Administrador
- **I want:** finalizar un torneo y registrar su ganador
- **So that:** cerrar el evento y otorgar el premio correspondiente (bono o premio monetario según tipo y elegibilidad)

Descripción (contexto)

Esta historia representa el cierre formal del torneo: cambiar estado a `Finalizado`, registrar quién ganó y aplicar la regla de premios. En `TorneoAmistoso`, se otorga un bono; en `TorneoCompetitivo`, se otorga premio monetario solo si el ganador es elegible (no empleado). Tu UML incorpora `GestorTorneos.finalizarTorneo(...)`, `TorneoAmistoso.otorgarBono(...)` y la regla “empleado no gana premio metálico” vía `InscripcionTorneo.elegiblePremioMetalico.`,

Entradas

- `torneoId`: String
- `ganadorLogin`: String

Validaciones

- Torneo existe y está `En_Curso` o `Programado` (según política).
- Ganador está inscrito en el torneo.
- Si competitivo:
 - Si ganador es empleado → **no premio metálico** (`esElegiblePremioMetalico=false`).,

Flujo

1. Admin selecciona “Finalizar torneo”.
2. Sistema ejecuta:

- a. `GestorTorneos.finalizarTorneo(admin, torneoId, ganador):`
`ResultadoTorneo` ,
3. Si torneo amistoso:
 - a. `TorneoAmistoso.otorgarBono(ganador):` `BonoDescubrimiento` (en tu UML aparece como *BonoDescubrimiento*, pero la clase concreta que tienes es *BonoTorneoAmistoso*; aquí conviene unificar nombres en implementación). ,
4. El torneo pasa a `EstadoTorneo.Finalizado`.

Salidas

- "Torneo finalizado. Ganador: X"
- Si amistoso: "Bono otorgado: código/valor"
- Si competitivo: "Premio metálico otorgado: monto" o "Ganador empleado: sin premio metálico".

Clases involucradas

- `GestorTorneos.finalizarTorneo(...)`
- `EstadoTorneo`
- `TorneoAmistoso.otorgarBono(...)`
- `InscripcionTorneo.elegiblePremioMetalico` ,

Criterios de aceptación

- Amistoso: se genera 1 bono para el ganador.
- Competitivo: solo si ganador es no-empleado y elegible.

CORRECCIONES para las de antes:

A) Historias NUEVAS derivadas del Proyecto 1 (sin repetir las de Torneos que ya tienes)

A continuación las organizo por tipo de usuario.

1) CLIENTE — Historias adicionales (Proyecto 1)

HU-C7 — Buscar juego en el catálogo y ver restricciones/disponibilidad

Estimación: TBD

Plantilla (tipo profe)

- **As a:** Cliente
- **I want:** buscar un juego por nombre y ver si está disponible para préstamo/venta y si es apto para mi mesa
- **So that:** elegir un juego que sí pueda jugar y que el café realmente tenga disponible

Descripción (contexto explícito)

Esta historia cubre la necesidad básica del Board Game Café: antes de pedir prestado o comprar, el cliente debe poder **consultar el catálogo**. El resultado debe indicar: categoría, rango de jugadores, edad mínima, si es "difícil", y si hay copia disponible para préstamo o venta. En tu UML esto se apoya en `Cafeteria.buscarJuego(nombre)` y en métodos del Juego como `getCopiaDisponible()`, `soportaNPersonas(n)` y `esAptoParaEdad(edadMinimaEnMesa)`.
[Entrega 2...on torneos | PDF], [uniandes-m...epoint.com]

Precondiciones

- Cliente autenticado (HU-T0).
- Datos cargados desde persistencia. [Entrega 2...on torneos | PDF]

Entradas (Consola)

- `nombreJuego`: `String`
- (Opcional) `personasEnMesa`: `int`, `hayNinos`: `boolean`, `hayJovenes`: `boolean` si el cliente quiere validar aptitud antes de reservar.

Validaciones

- El nombre no puede ser vacío.
- Si el juego no existe, informar y permitir reintento.

Flujo principal

1. Cliente digita nombre del juego.
2. Sistema ejecuta `Cafeteria.buscarJuego(nombre)`. [Entrega 2...on torneos | PDF]
3. Si existe, muestra atributos del Juego y disponibilidad:

- a. Préstamo: `Juego.getCopiaDisponible()` / `getCopiasPrestamo()` [\[Entrega 2...on torneos | PDF\]](#)
 - b. Venta: `Juego.estaDisponibleParaVenta()` [\[Entrega 2...on torneos | PDF\]](#)
- 4. Si el cliente dio datos de mesa, valida:
 - a. `Juego.soportaNPersonas(n)` y `Juego.esAptoParaEdad(...)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas/Resultados

- Ficha del juego + "Disponible para préstamo: Sí/No" + "Disponible para venta: Sí/No".
- Si no es apto, "No apto por edad o número de jugadores".

Clases involucradas (UML)

- `Cafeteria.buscarJuego(...)` [\[Entrega 2...on torneos | PDF\]](#)
- `Juego` (métodos de disponibilidad y aptitud) [\[Entrega 2...on torneos | PDF\]](#)

Criterios de aceptación

- Si el juego no existe, se reporta claramente.
- Si no hay copias disponibles, se muestra "no disponible" aunque esté en catálogo.

HU-C8 — Reservar mesa (validando capacidad del café)

Estimación: TBD

Plantilla (tipo profe)

- **As a:** Cliente
- **I want:** reservar/ocupar una mesa indicando cuántas personas y si hay niños/jóvenes
- **So that:** el sistema valide capacidad y luego permita préstamos de juegos adecuados

Descripción (contexto explícito)

En el P1 se exige que para préstamos de clientes exista mesa, que el café tenga capacidad, y que se capture si hay niños/jóvenes para restringir juegos y bebidas. Tu UML lo cubre con `Cliente.reservarMesa(...)`, `Cafeteria.hayCapacidad(...)`, `Cafeteria.getMesaDisponible(...)` y `Mesa.ocupar(...)`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Precondiciones

- Cliente autenticado.

- El cliente no debe estar ocupando otra mesa (regla de negocio recomendada).

Entradas (Consola)

- personas: int
- hayNinos: boolean
- hayJovenes: boolean

Validaciones

- personas > 0
- Cafeteria.hayCapacidad(personas) debe ser true. [\[Entrega 2...on torneos | PDF\]](#)
- Debe existir una mesa disponible con capacidad:
Cafeteria.getMesaDisponible(personas). [\[Entrega 2...on torneos | PDF\]](#)

Flujo principal

1. Cliente ingresa personas y banderas.
2. Sistema verifica capacidad.
3. Sistema asigna mesa disponible y la ocupa con Mesa.ocupar(...). [\[Entrega 2...on torneos | PDF\]](#)
4. Retorna la mesa al cliente (Cliente.reservarMesa(...)). [\[Entrega 2...on torneos | PDF\]](#)

Salidas/Resultados

- "Mesa asignada: idMesa = X"
- En caso de falla: "No hay capacidad/No hay mesa disponible".

Clases UML

- Cliente.reservarMesa(...) [\[Entrega 2...on torneos | PDF\]](#)
- Cafeteria.hayCapacidad, Cafeteria.getMesaDisponible [\[Entrega 2...on torneos | PDF\]](#)
- Mesa.ocupar [\[Entrega 2...on torneos | PDF\]](#)

Criterios de aceptación

- Si el café está lleno, la reserva se rechaza claramente.
- La mesa queda marcada como ocupada.

HU-C9 — Solicitar préstamo de juego (cliente) con reglas de mesa/edad/jugadores/dificultad

Estimación: TBD

Plantilla (tipo profe)

- **As a:** Cliente
- **I want:** solicitar un préstamo de un juego para mi mesa
- **So that:** pueda jugar mientras consumo, cumpliendo reglas de edad y número de jugadores

Descripción (contexto explícito)

Esta historia implementa el préstamo para clientes: **requiere mesa**, valida si el juego sirve por número de personas y edad mínima, valida disponibilidad de copias, y considera juegos "difíciles" (si hay mesero capacitado o se presta bajo advertencia). En tu UML, el préstamo se crea con `Cliente.solicitarPréstamo(copia, mesa)` y se controla con `Juego.getCopiaDisponible()`, `Juego.esAptoParaEdad(...)`, `Juego.soportaNPersonas(n)` y `CopiaPréstamo.prestar()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Precondiciones

- Cliente con mesa asignada (HU-C8).
- Cliente no excede el máximo de juegos prestados simultáneamente (P1 dice hasta 2; tu UML sugiere control con `juegosReservados` en `Cliente`). [\[Entrega 2...on torneos | PDF\]](#)

Entradas

- `nombreJuego`: `String` (o `id/copia`)
- Confirmación si es "difícil" y no hay mesero disponible (si lo manejas en UI)

Validaciones

- Existe copia disponible: `Juego.getCopiaDisponible()` no null. [\[Entrega 2...on torneos | PDF\]](#)
- Apto por jugadores: `Juego.soportaNPersonas(personasMesa)` [\[Entrega 2...on torneos | PDF\]](#)
- Apto por edad: `Juego.esAptoParaEdad(edadMinimaEnMesa)` (según `hayNinos/hayJovenes`) [\[Entrega 2...on torneos | PDF\]](#)
- Regla de máximo 2 préstamos por cliente (nota/constraint que conviene explicitar en implementación).

Flujo

1. Cliente elige juego.
2. Sistema obtiene copia disponible.
3. Sistema valida aptitud por mesa.
4. Si "difícil", (opcional) consulta si hay mesero que pueda enseñar (`Mesero.puedeEnsenar(j)`) o muestra advertencia. [\[Entrega 2...on torneos | PDF\]](#)
5. Se crea `Prestamo` y se llama `CopiaPrestamo.prestar()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Préstamo creado: copia X, inicio Y"
- O "No disponible / No apto / Excede máximo de préstamos"

Clases UML

- `Cliente.solicitarPrestamo(copia, mesa): Prestamo` [\[Entrega 2...on torneos | PDF\]](#)
- `Juego.getCopiaDisponible, esAptoParaEdad, soportaNPersonas` [\[Entrega 2...on torneos | PDF\]](#)
- `CopiaPrestamo.prestar` [\[Entrega 2...on torneos | PDF\]](#)
- `Prestamo (inicio/fin/estado)` [\[Entrega 2...on torneos | PDF\]](#)

Criterios de aceptación

- Si el juego no es apto, el préstamo se bloquea.
- Si no hay copias, se rechaza aunque el juego esté en catálogo.

HU-C10 — Devolver juego prestado y finalizar préstamo

Estimación: TBD

Plantilla

- **As a:** Cliente
- **I want:** devolver el juego prestado al finalizar mi estadía
- **So that:** el juego quede disponible para otros y el historial quede registrado

Descripción

P1 exige devolución y registro en historial. En tu UML, el cliente devuelve con `Cliente.devolverJuego(p)` y el préstamo puede finalizarse con `Prestamo.finalizar()` y la

copia se marca disponible con `CopiaPrestamo.devolver()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Selección del préstamo activo (por id o listando préstamos del cliente)

Validaciones

- El préstamo pertenece al cliente.
- El préstamo no está ya finalizado.

Flujo

1. Cliente selecciona préstamo.
2. Sistema llama `Prestamo.finalizar()` y `CopiaPrestamo.devolver()`. [\[Entrega 2...on torneos | PDF\]](#)
3. Se actualiza historial ("historialPrestamos" en relación desde Cafetería). [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Salidas

- "Devolución exitosa. Duración: X horas" (`Prestamo.getDuracionHoras()`). [\[Entrega 2...on torneos | PDF\]](#)

Criterios de aceptación

- La copia queda disponible.
- El préstamo queda registrado para consulta del administrador.

HU-C11 — Comprar productos de cafetería (bebidas/pastelería) con impuestos y propina

Estimación: TBD

Plantilla

- **As a:** Cliente
- **I want:** comprar productos del menú del café (bebidas, pastelería)
- **So that:** se registre una venta con impuestos de cafetería y propina configurable

Descripción

P1 exige ventas con rubros, impuestos y propina. Tu UML soporta ventas con Venta (subtotal/impuestos/propina/total) y cálculo con `calcularImpuestosTotales`, `setPropina`, etc. Además, los ítems se modelan con `ItemVenta` y el producto implementa `ProductoVendible.getTasaImpuesto()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Lista de ítems: producto + cantidad
- Propina (%) (opcional; default 10)
- Mesa (si es consumo en mesa)

Validaciones

- Productos existen en `Cafeteria.productos`. [\[Entrega 2...on torneos | PDF\]](#)
- Restricciones bebidas:
 - Si bebida alcohólica y hay menores, bloquear (regla de negocio del enunciado; tu UML tiene flags `Bebida.esAlcoholica` y mesa tiene `hayNinos/hayJovenes`). [\[Entrega 2...on torneos | PDF\]](#)
 - Bebida caliente no debe coexistir con juego de acción (puedes validar con `Mesa.puedeRecibirBebidaCaliente()` y `Mesa.puedeRecibirJuegoAccion()`). [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Cliente selecciona productos y cantidades.
2. Sistema crea Venta con `itemsVenta` y marca `ventaCafeteria = true`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)
3. Calcula subtotal/impuestos/propina/total. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Factura: subtotal, impuestos, propina, total.

Clases UML

- Venta + ItemVenta + ProductoVendible + Bebida + Pasteleria + Mesa [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Criterios de aceptación

- Se registra venta completa con rubros.
- Se bloquean combinaciones peligrosas (alcohol con menores; caliente con acción).

HU-C12 — Consultar alérgenos de productos de pastelería antes de comprar

Estimación: TBD

Plantilla

- **As a:** Cliente
- **I want:** ver alérgenos de una pastelería
- **So that:** tomar una decisión informada por salud antes de comprar

Descripción

Tu UML incluye `Pasteleria.alergenos + getAlergenos()` y además `Cafeteria.consultarAlergenos(p: Pasteleria)`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Selección de producto `Pasteleria`

Validaciones

- El producto seleccionado es de tipo pastelería.

Flujo

1. Cliente selecciona pastelería.
2. Sistema muestra `Cafeteria.consultarAlergenos(p)` o `p.getAlergenos()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Lista de alérgenos.

Criterios de aceptación

- Siempre se muestra la lista (aunque esté vacía).

HU-C13 — Comprar juego de mesa (tienda) y registrar venta con IVA

Estimación: TBD

Plantilla

- **As a:** Cliente
- **I want:** comprar juegos de mesa en una transacción
- **So that:** el sistema registre la venta con impuestos (IVA) y puntos de fidelidad

Descripción

P1 distingue inventario de venta vs préstamo. En tu UML, el juego tiene copias para venta (CopiaVenta) y la venta se registra via `Cliente.realizarCompra(items, codigoDesc)` o métodos de venta (`GestorVentas.registrarVenta`). También Venta genera puntos (`puntosGenerados`) con `calcularPuntosGenerados()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Lista de `ItemVenta` (juegos/productos vendibles) + cantidades
- `codigoDesc` opcional

Validaciones

- Copias disponibles para venta.
- Cálculo de impuestos según tasa del producto (`ProductoVendible.getTasaImpuesto`). [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Cliente arma items.
2. `Cliente.realizarCompra(items, codigoDesc)`: Venta. [\[Entrega 2...on torneos | PDF\]](#)
3. `Venta.calcular...` y actualización de puntos fidelidad. [\[Entrega 2...on torneos | PDF\]](#)
4. `GestorVentas.registrarVenta(venta)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Factura + puntos generados.

Criterios de aceptación

- La venta queda registrada con fecha y rubros.
- Los puntos aumentan según el total (regla del enunciado; en UML está el soporte de puntos). [\[Entrega 2...on torneos | PDF\]](#)

HU-C14 — Redimir puntos de fidelidad como descuento

Estimación: TBD

Plantilla

- **As a:** Cliente
- **I want:** usar mis puntos de fidelidad como descuento en una compra
- **So that:** reducir el total a pagar usando el programa de fidelidad

Descripción

Tu UML trae `Cliente.usarPuntosFidelidad(puntos): double` y `Venta` mantiene `puntosGenerados`. Esto soporta la redención como descuento en compras futuras. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `puntosAUsar: int`

Validaciones

- `puntosAUsar > 0`
- `puntosAUsar <= puntosFidelidad del cliente`. [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Cliente indica puntos a usar durante compra.
2. Sistema ejecuta `Cliente.usarPuntosFidelidad(puntos)` para obtener descuento. [\[Entrega 2...on torneos | PDF\]](#)
3. Se recalcula total.

Salidas

- "Descuento aplicado: X"
- "Puntos restantes: Y"

Criterios de aceptación

- No permite usar más puntos de los que tiene.

2) EMPLEADO — Historias adicionales (Proyecto 1)

HU-E7 — Alquilar/jugar un juego como empleado (sin mesa) cuando no está de turno

Estimación: TBD

Plantilla

- **As a:** Empleado
- **I want:** pedir prestado un juego para uso personal sin mesa
- **So that:** poder practicar/usar juegos cuando no estoy en turno y no hay clientes que atender

Descripción

P1 permite préstamos a empleados bajo condiciones. En tu UML, el empleado puede alquilar juegos con `Empleado.alquilarJuego(copia)` y validar turno con `Empleado.estaEnTurno()` / atributo `estaDeTurno`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `nombreJuego` o selección de copia disponible

Validaciones

- `Empleado.estaEnTurno() == false`. [\[Entrega 2...on torneos | PDF\]](#)
- Copia disponible para préstamo: `CopiaPrestamo.estaDisponible()`. [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Empleado selecciona juego/copia.
2. Sistema valida no esté de turno.
3. Crea préstamo (sin mesa) vía `Empleado.alquilarJuego(copia)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Préstamo a empleado creado"

Criterios

- Si está en turno, se rechaza.

HU-E8 — Sugerir un nuevo platillo para el menú

Estimación: TBD

Plantilla

- **As a:** Empleado (cocinero o mesero)
- **I want:** sugerir un platillo nuevo
- **So that:** el administrador lo evalúe y el menú pueda mejorar

Descripción

P1 contempla sugerencias de menú de empleados. En tu UML existe `Empleado.sugerirPlato(desc)` que crea `SugerenciaMenu`, y el `Administrador` puede aprobarla (`aprobarSugerenciaMenu`). [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `descripcion: String`

Validaciones

- Descripción no vacía.

Flujo

1. Empleado envía sugerencia: `Empleado.sugerirPlato(desc)`. [\[Entrega 2...on torneos | PDF\]](#)
2. Se guarda la `SugerenciaMenu` en estado "Pendiente" (regla recomendada; tu UML tiene estado). [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Sugerencia registrada y pendiente de aprobación".

Criterios

- Queda visible para administración/aprobación.

HU-E9 — Empleado realiza compra con su código de descuento

Estimación: TBD

Plantilla

- **As a:** Empleado
- **I want:** usar mi código de descuento al comprar
- **So that:** obtener el beneficio de empleado y registrar la venta correctamente

Descripción

P1 indica descuentos 20% empleado y 10% para otros con quienes comparte (no acumulable). En tu UML el empleado tiene `codigoDescuento` y la venta permite `aplicarDescuento(codigo)`.

[Entrega 2...on torneos | PDF], [uniandes-m...epoint.com]

Entradas

- Items de compra
- `codigoDescuento` (auto: el del empleado)

Validaciones

- Código válido (regla en lógica, aunque el UML no detalla verificación).
- No acumulable con otros descuentos (nota/constraint que conviene mantener coherente con tu nota del bono también). [Entrega 2...on torneos | PDF]

Flujo

1. Empleado crea compra: `Empleado.realizarCompra(items)`. [Entrega 2...on torneos | PDF]
2. Sistema aplica descuento: `Venta.aplicarDescuento(codigo)`. [Entrega 2...on torneos | PDF]

Salidas

- Factura con descuento aplicado.

Criterios

- Si se aplica descuento, no permite aplicar otro descuento adicional.

HU-E10 — Mesero gestiona “juegos difíciles”: registrar que sabe enseñarlos

Estimación: TBD

Plantilla

- **As a:** Mesero
- **I want:** registrar que conozco/puedo enseñar un juego difícil
- **So that:** el sistema pueda identificar si hay alguien que enseñe reglas al prestarlo

Descripción

P1 define juegos “difíciles” que idealmente requieren mesero capacitado. Tu UML tiene `Mesero.agregarJuegoConocido(j)` y `Mesero.puedeEnsenar(j)`, además de relación `Mesero – juegosConocidos`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Selección de Juego

Validaciones

- Juego existe.

Flujo

1. Mesero selecciona juego.
2. Ejecuta `Mesero.agregarJuegoConocido(j)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Confirmación: “Juego agregado a conocimientos”.

Criterios

- El mismo juego no se duplica en lista.

3) ADMINISTRADOR — Historias adicionales (Proyecto 1)

HU-A8 — Construir y modificar turnos semanales de empleados

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** configurar y modificar turnos semanales de empleados
- **So that:** asegurar operación mínima (1 cocinero, 2 meseros) y controlar disponibilidad

Descripción

P1 exige que el administrador pueda construir y modificar turnos. Tu UML incluye Semana con actualizarTurno(día, nuevoTurno) y acceso por getTurnoDelDia(día), además de DiaTurno.asignar/liberar/setAprobado. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- empleadoLogin
- día: DiaSemana
- Acción: asignar / liberar / aprobar

Validaciones

- Verificar mínimo de empleados en el día: Cafeteria.validarMinimoEmpleados(día). [\[Entrega 2...on torneos | PDF\]](#)
- Solo rol admin.

Flujo

1. Admin selecciona empleado + día.
2. Modifica turno con Semana.actualizarTurno(...) y DiaTurno.asignar/liberar. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Turno actualizado".

Criterios

- No permite configuración que viole el mínimo.

HU-A9 — Aprobar/Rechazar sugerencias de menú

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** aprobar o rechazar sugerencias de menú
- **So that:** controlar el menú oficial del café

Descripción

Complementa HU-E8. En tu UML el administrador tiene `aprobarSugerenciaMenu(s)`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Selección de `SugerenciaMenu`

Validaciones

- Sugerencia existe y está "pendiente".

Flujo

1. Admin revisa sugerencias.
2. Aprueba con `Administrador.aprobarSugerenciaMenu(s)` o la rechaza (si lo implementas como cambio de estado). [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Estado actualizado.

Criterios

- Solo admin puede aprobar.

HU-A10 — Agregar un producto nuevo al menú (bebida/pastelería)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** agregar productos nuevos al menú
- **So that:** el sistema permita venderlos en la cafetería

Descripción

P1 permite al administrador agregar platillos. Tu UML tiene `Administrador.agregarProductoMenu(p: ProductoComestible)` y el menú está en `Cafeteria.productos`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Tipo: Bebida o Pastelería
- Datos del producto (nombre, precioBase, flags de bebida o alérgenos de pastelería)

Validaciones

- Nombre no vacío; precio > 0.

Flujo

1. Admin crea objeto Bebida o Pastelería.
2. Llama `Administrador.agregarProductoMenu(p)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Producto agregado al menú".

HU-A11 — Comprar/reabastecer inventario de juegos (préstamo o venta)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** comprar juegos y aumentar inventario para préstamo o venta
- **So that:** asegurar disponibilidad para clientes y ventas

Descripción

P1 exige que admin pueda reabastecer inventario de préstamo/venta. En tu UML, Administrador.comprarJuegos(j, cant, tipo) y Juego.agregarCopiaPrestamo/agregarCopiaVenta. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- Juego (existente o a crear, según diseño)
- cant: int
- tipo: String ("prestamo" o "venta")

Validaciones

- cant > 0
- tipo válido

Flujo

1. Admin elige juego y cantidad.
2. Ejecuta Administrador.comprarJuegos(j, cant, tipo). [\[Entrega 2...on torneos | PDF\]](#)
3. Se agregan copias al juego.

Salidas

- "Inventario actualizado".

HU-A12 — Mover un juego del inventario de préstamo al inventario de venta

Estimación: TBD

Plantilla

- **As a:** Administrador

- **I want:** mover una copia de préstamo al inventario de venta
- **So that:** convertir unidades en stock vendible según necesidades del negocio

Descripción

P1 menciona mover inventario; tu UML tiene `Administrador.moverJuegoAVenta(c: CopiaPréstamo)`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPréstamo: String`

Validaciones

- La copia existe y no está prestada (debe estar disponible).

Flujo

1. Admin selecciona copia.
2. Ejecuta `Administrador.moverJuegoAVenta(copiaPréstamo)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Copia movida a inventario de venta".

HU-A13 — Reparar un juego (reemplazar copia de préstamo por una de venta)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** reparar un juego dañado
- **So that:** mantener el inventario de préstamo funcional sin perder trazabilidad

Descripción

P1 define "reparar" como reemplazar copia de préstamo por una de venta. Tu UML contiene `Administrador.repararJuego(c: CopiaPréstamo)` y `CopiaPréstamo.reparar()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPrestamo`

Validaciones

- Copia existe.
- No debe estar prestada.

Flujo

1. Admin selecciona copia.
2. Ejecuta `Administrador.repararJuego(copia)` y/o `copia.reparar()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Juego reparado / copia reemplazada".

HU-A14 — Dar un juego por robado (marcar desaparecida una copia)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** marcar un juego como robado/desaparecido cuando no se devuelve
- **So that:** mantener inventario consistente y asumir pérdida

Descripción

P1 exige marcar "desaparecido". Tu UML tiene `Administrador.darJuegoPorRobado(c: CopiaPrestamo)` y `CopiaPrestamo.marcarDesaparecida()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPrestamo`

Validaciones

- Copia existe.

- (Regla recomendada) préstamo vencido: `Prestamo.estaVencido()` antes de permitir. [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Admin selecciona copia y préstamo asociado.
2. Marca como robado: `Administrador.darJuegoPorRobado(copia) → copia.marcarDesaparecida()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Copia marcada como desaparecida".

HU-A15 — Ver historial detallado de un juego (préstamos y estado)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** ver el historial de un juego (cuándo, cuántas veces, a quién se prestó, estado)
- **So that:** auditar inventario y tomar decisiones (reparar, reabastecer, retirar)

Descripción

P1 dice que debe existir historial completo. Tu UML incluye `Administrador.verHistorialJuego(j): String`, `CopiaPrestamo.vecesPrestado`, y relación de Cafeteria con `historialPrestamos`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `nombreJuego` o selección del juego

Validaciones

- Juego existe.

Flujo

1. Admin selecciona juego.
2. Ejecuta `Administrador.verHistorialJuego(j)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Reporte en texto con eventos de préstamo y estado.

Criterios

- Muestra datos suficientes para auditoría.

HU-A12 — Mover un juego del inventario de préstamo al inventario de venta

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** mover una copia de préstamo al inventario de venta
- **So that:** convertir unidades en stock vendible según necesidades del negocio

Descripción

P1 menciona mover inventario; tu UML tiene `Administrador.moverJuegoAVenta(c: CopiaPréstamo)`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPréstamo: String`

Validaciones

- La copia existe y no está prestada (debe estar disponible).

Flujo

1. Admin selecciona copia.
2. Ejecuta `Administrador.moverJuegoAVenta(copiaPréstamo)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Copia movida a inventario de venta".

HU-A13 — Reparar un juego (reemplazar copia de préstamo por una de venta)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** reparar un juego dañado
- **So that:** mantener el inventario de préstamo funcional sin perder trazabilidad

Descripción

P1 define "reparar" como reemplazar copia de préstamo por una de venta. Tu UML contiene `Administrador.repararJuego(c: CopiaPrestamo)` y `CopiaPrestamo.reparar()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPrestamo`

Validaciones

- Copia existe.
- No debe estar prestada.

Flujo

1. Admin selecciona copia.
2. Ejecuta `Administrador.repararJuego(copia)` y/o `copia.reparar()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Juego reparado / copia reemplazada".

HU-A14 — Dar un juego por robado (marcar desaparecida una copia)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** marcar un juego como robado/desaparecido cuando no se devuelve
- **So that:** mantener inventario consistente y asumir pérdida

Descripción

P1 exige marcar "desaparecido". Tu UML tiene `Administrador.darJuegoPorRobado(c: CopiaPrestamo)` y `CopiaPrestamo.marcarDesaparecida()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPrestamo`

Validaciones

- Copia existe.
- (Regla recomendada) préstamo vencido: `Prestamo.estaVencido()` antes de permitir. [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Admin selecciona copia y préstamo asociado.
2. Marca como robado: `Administrador.darJuegoPorRobado(copia) → copia.marcarDesaparecida()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Copia marcada como desaparecida".

HU-A15 — Ver historial detallado de un juego (préstamos y estado)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** ver el historial de un juego (cuándo, cuántas veces, a quién se prestó, estado)
- **So that:** auditar inventario y tomar decisiones (reparar, reabastecer, retirar)

Descripción

P1 dice que debe existir historial completo. Tu UML incluye `Administrador.verHistorialJuego(j): String`, `CopiaPrestamo.vecesPrestado`, y relación de Cafeteria con `historialPrestamos`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `nombreJuego` o selección del juego

Validaciones

- Juego existe.

Flujo

1. Admin selecciona juego.
2. Ejecuta `Administrador.verHistorialJuego(j)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Reporte en texto con eventos de préstamo y estado.

Criterios

- Muestra datos suficientes para auditoría.

HU-A12 — Mover un juego del inventario de préstamo al inventario de venta

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** mover una copia de préstamo al inventario de venta
- **So that:** convertir unidades en stock vendible según necesidades del negocio

Descripción

P1 menciona mover inventario; tu UML tiene `Administrador.moverJuegoAVenta(c: CopiaPrestamo)`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPrestamo: String`

Validaciones

- La copia existe y no está prestada (debe estar disponible).

Flujo

1. Admin selecciona copia.
2. Ejecuta `Administrador.moverJuegoAVenta(copiaPrestamo)`. [[Entrega 2...on torneos | PDF](#)]

Salidas

- "Copia movida a inventario de venta".

HU-A13 — Reparar un juego (reemplazar copia de préstamo por una de venta)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** reparar un juego dañado
- **So that:** mantener el inventario de préstamo funcional sin perder trazabilidad

Descripción

P1 define "reparar" como reemplazar copia de préstamo por una de venta. Tu UML contiene `Administrador.repararJuego(c: CopiaPrestamo)` y `CopiaPrestamo.reparar()`. [[Entrega 2...on torneos | PDF](#)], [[uniandes-m...epoint.com](#)]

Entradas

- `idCopiaPrestamo`

Validaciones

- Copia existe.
- No debe estar prestada.

Flujo

1. Admin selecciona copia.
2. Ejecuta `Administrador.repararJuego(copia)` y/o `copia.reparar()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- "Juego reparado / copia reemplazada".

HU-A14 — Dar un juego por robado (marcar desaparecida una copia)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** marcar un juego como robado/desaparecido cuando no se devuelve
- **So that:** mantener inventario consistente y asumir pérdida

Descripción

P1 exige marcar "desaparecido". Tu UML tiene `Administrador.darJuegoPorRobado(c: CopiaPrestamo)` y `CopiaPrestamo.marcarDesaparecida()`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `idCopiaPrestamo`

Validaciones

- Copia existe.
- (Regla recomendada) préstamo vencido: `Prestamo.estaVencido()` antes de permitir. [\[Entrega 2...on torneos | PDF\]](#)

Flujo

1. Admin selecciona copia y préstamo asociado.
2. Marca como robado: `Administrador.darJuegoPorRobado(copia) → copia.marcarDesaparecida()`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- “Copia marcada como desaparecida”.

HU-A15 — Ver historial detallado de un juego (préstamos y estado)

Estimación: TBD

Plantilla

- **As a:** Administrador
- **I want:** ver el historial de un juego (cuándo, cuántas veces, a quién se prestó, estado)
- **So that:** auditar inventario y tomar decisiones (reparar, reabastecer, retirar)

Descripción

P1 dice que debe existir historial completo. Tu UML incluye `Administrador.verHistorialJuego(j): String`, `CopiaPrestamo.vecasPrestado`, y relación de `Cafeteria` con `historialPrestamos`. [\[Entrega 2...on torneos | PDF\]](#), [\[uniandes-m...epoint.com\]](#)

Entradas

- `nombreJuego` o selección del juego

Validaciones

- Juego existe.

Flujo

1. Admin selecciona juego.
2. Ejecuta `Administrador.verHistorialJuego(j)`. [\[Entrega 2...on torneos | PDF\]](#)

Salidas

- Reporte en texto con eventos de préstamo y estado.

Criterios

- Muestra datos suficientes para auditoría.